NLU Project – Part 2: NLU

Andrea Giampietro (258237)

1. Introduction

This report covers Part 2 of the NLU project, focusing on intent classification and slot filling on the ATIS dataset. Intent classification identifies the overall purpose of a sentence, while slot filling labels tokens with semantic roles. The project has two parts:

- In Part A, enhancements are made to the initial LSTM baseline by incorporating bidirectional layers and dropout for improved robustness.
- Part B focuses on leveraging a pre-trained BERT model, which is fine-tuned to perform both intent classification and slot filling simultaneously.

2. Methodologies

All modifications were introduced incrementally, with performance monitored at each stage via loss. The overall loss was calculated as the sum of two separate cross-entropy losses: one for intent classification and one for slot filling. Each experiment was repeated five times, with the best model from each run saved for testing. The reported results represent the average performance across these five runs. Early stopping with a patience of 3 epochs was applied consistently. Additionally, a test mode was implemented to re-evaluate saved models and verify the logged outcomes.

In Part A, the baseline LSTM model was enhanced with the following:

1. **Bidirectionality:** Activated using PyTorch's Bidirectional=True flag in the LSTM layer. Subsequent layer dimensions were adjusted to accommodate the doubled hidden size.
2. **Dropout:** Added after the LSTM layer and before both the slot filling and intent classification heads to reduce overfitting.

For the BERT-based implementation, I adopted a joint learning framework utilizing the bert-base-uncased pre-trained model. The architecture leverages BERT's bidirectional contextual representations through a dual-head approach: sentence-level classification for intent detection using the [CLS] token's pooled representation, and sequence labeling for slot filling using individual token embeddings from BERT's final layer.

The primary technical challenge arose from BERT's WordPiece tokenization mechanism, which decomposes input words into subword units. This process creates a mismatch between the original word-level slot annotations and BERT's token-level representations. For instance, compound words like "downtown" may be segmented into ["down", "##town"], requiring careful label propagation strategies.

My solution implements a specialized preprocessing pipeline that maintains label consistency across subword boundaries. During tokenization with BERT's native tokenizer, an alignment algorithm that identifies subword continuations through their distinctive "##" prefix pattern was developed. The original slot label is propagated from the initial subword to all subsequent fragments of the same word, ensuring semantic coherence. Special boundary tokens ([CLS], [SEP]) receive neutral padding labels to exclude them from slot-level computations.

The training procedure incorporates attention-based masking to focus learning exclusively on meaningful tokens. Loss computation for slot filling utilizes only positions with active attention weights, preventing the model from learning spurious patterns from padding elements. This selective training strategy significantly improves convergence stability and final performance.

Model evaluation follows established protocols, with slot filling performance measured using token-level F1 scores computed via CoNLL-style evaluation metrics. Intent classification accuracy is calculated using standard multi-class classification measures. Both metrics exclude artificial padding positions to ensure fair performance assessment.

3. Results

A thorough hyperparameter tuning process was conducted to maximize model effectiveness on the joint intent classification and slot filling task. The best-performing models and their corresponding configurations are summarized below.

Part A: Both key enhancements — introducing bidirectionality and adding dropout layers — demonstrated clear improvements compared to the baseline LSTM model.

- The baseline LSTM achieved a slot filling F1 score of **0.926** and an intent classification accuracy of **0.929**.
- Enabling bidirectionality significantly boosted results, with slot F1 increasing to **0.939** and intent accuracy to **0.953**.
- Adding dropout on top of the bidirectional model led to further gains, yielding a slot F1 of **0.943** and intent accuracy of **0.950**.

Regarding hyperparameters, models with moderate hidden and embedding sizes showed more stable performance, while dropout probabilities around 0.3 consistently delivered the best balance between regularization and learning capacity. Learning rates were carefully tuned to avoid under- or overfitting, with the most effective values falling within a low range typical for recurrent architectures.

Part B: Fine-tuning the pre-trained **BERT** model resulted in better performance compared to the LSTM architecture used in Part A. Experimental results showed that small variations in learning rate, dropout probability, and batch size had only a marginal impact on performance, suggesting that BERT is less sensitive to hyperparameter tuning than the LSTM model. The best configuration, with a dropout probability of 0.1, a learning rate of 2×10^{-5}, and batch sizes of 16/64/64, achieved a **Slot F1 score of 0.890** and an **Intent Accuracy of 0.969**. These results represent an improvement over the baseline model and confirm the robustness of transformer-based architectures for joint intent detection and slot filling tasks.

Why BERT underperformed compared to LSTM

- **Subword tokenization:** WordPiece splits words, complicating exact label alignment and introducing noise in slot annotations.
- **Limited hyperparameter tuning:** The search space may have been too narrow to fully optimize BERT.
- **Dataset size:** The small, domain-specific ATIS dataset may favor LSTMs that learn task-specific patterns directly.
- **Evaluation sensitivity:** Token-level F1 penalizes subword label mismatches, affecting BERT more due to its tokenization.

Table 1: Part 2A – Model Variants with Hyperparameters and Test Metrics

Model Configuration	Learning Rate	Batch Size (Test)	Hidden/Embedding Size	Dropout	Slot F1 Score	Intent Accuracy
ModelIAS (vanilla LSTM)	0.001	64	200 / 300	–	0.9256	0.9295
Bidirectional ModelIAS	0.001	64	200 / 300	–	0.9390	0.9530
+ Dropout	0.001	64	200 / 300	0.3	**0.9432**	0.9496